

# Midterm Practice Problems

---

See below some practice problems to help you polish your skill.

This is not a test and solutions you create are not marked. The problems below are not an indication of the midterm test question types or difficulty.

## Templates

- Create a family of functions (a template function) that checks how many times a certain item appears in an array. The function should return the number of occurrences. Specialize for `std::string` to check how many times the first letter of a string appears in all strings from in the array.
- Create a family of functions (a template function) that returns how many elements in an array are smaller than a specific value. Specialize for `std::string` to compare by string length.
- Create a family of functions (a template function) that replaces all elements from an array that have a specific value with another specific value. The value to replace and the new value are received as parameters. Specialize for `char*` to compare the strings by content; if the new value requires more characters than the old value, raise an exception to inform the client.
- Create a family of functions (a template function) that calculates the sum of all elements from an array received as parameter. Specialize for `std::string` to concatenate the strings, separated by comma ( , ).
- Create a family of functions (a template function) that checks if an certain value exists in an array. Both the value and the array are parameters. Specialize for `char*` to compare the strings by content, not by address.
- Create a family of functions (a template function) that counts how many elements in an array match a certain condition. The array and the condition are received as parameters. The condition is a lambda expression.
- Create a family of functions (a template function) that counts how many elements of an array are smaller than a certain value and grater than another value. Specialize for `char*` to compare by string length.
- Create a family of functions (a template function) that searches in an array an element that matches a certain predicate. The predicate is received as parameter and is a lambda expression. Return the address of the first element that matches, or null if no match is found.

- Create a family of functions (a template function) that returns `true` if all items in an array received as a parameter match a predicate (also received as a parameter). If at least one item doesn't match, return `false`.
- Create a family of functions (a template function) that finds the second biggest element in an array received as a parameter. If there is no such element (all elements are equal, or the array contains a single element), raise an exception to inform the client. Specialize for `char*` to compare the strings by content.

A type that uses the templates above must include in its definition certain functions and/or operators. Identify each function and/or operator that your template assumes is de-fined. You may do so in the form of an exact prototype or an English descriptive phrase. To test that you identified all assumptions, create a custom type and make it work with your template (add the special operations and print messages with `std::clog` so you can trace which one is used).

## Recursive Functions

- Define a recursive function that receives an integer `n` as a parameter and prints to screen:

```
.                <- 1 dot
..              <- 2 dots
...            <- 3 dots
....          <- 4 dots
.....        <- 5 dots
              <- ...
.....        <- n dots
```

- Define a recursive function that receives an integer `n` as a parameter and calculates the formula:  $1 + 2 \times 2 + 3 + 2 \times 4 + 5 + 2 \times 6 + \dots$  up to `n`.
- Define a recursive function that receives an integer `n` as a parameter and calculates the formula  $(0 \times 1) + (1 \times 2) + (2 \times 3) + (3 \times 4) + \dots (n-1) \times n$ .
- Define a recursive function that receives an `std::string` as a parameter and counts how many times the letter `a` appears.
- Define a recursive function that receives an array of `double` as parameter and calculates the sum of all elements from the array.
- Define a recursive function that receives an array of integers as parameter and returns how many even numbers are in the array.
- Define a recursive function that receives an `std::string` as a parameter and replaces all spaces with a dot ( `.` ).
- Define a recursive function that receives an array of floating point numbers in single precision as

parameter and replaces each negative value with the absolute value.

- Define a recursive function that receives an integer as a parameter and finds out how many times the number can be divided by 2 (without a remainder).
- Define a recursive function that receives an array of integers as parameter and prints them to screen separated by comma ( , ).
- Define a recursive function that receives an array of integers as parameter and returns to the client a string containing all the numbers separated by comma ( , ).

For all **recursive** functions created above, write the corresponding **iterative** one and measure the time each one needs to finish (if the functions are too fast, call each one a 1000 times or more until the measured times are meaningful).

## Function Objects

- Create a functor (a function object) that validates a password. At instantiation, the function object should receive from the client
  - a set of characters -> the valid password must contain at least one (i.e., special characters like `#!@$$%^&*( )`);
  - a set of characters that should not be in the password (i.e., space, new line, etc)

Through modifiers, the client should be able to specify if the password must contain at least:

- one lower case letter (by default, yes)
- one upper case letter (by default, yes)
- a number (by default, no)
- a minimum number of characters (by default, no; if yes, then the client must specify this minimum).

The functor should return true if a password is valid; false otherwise;

- Create a functor (a function object) that removes the spaces from the beginning/end of any string. An instance should be able to keep track of how many strings were cleared and how many spaces were removed; and provide these values to a client through a query.
- Create a functor (a function object) that checks if a string is a valid name. A valid name is a string that contains exactly 2 words, each one starting with an upper case letter. Each instance should remember the last 10 valid strings and the last 10 invalid strings; provide a query to print these strings to the screen.
- Create a functor (a function object) that removes duplicates from an array of numbers. At instantiation, the client should be able to specify a numeric value; any two numbers whose difference is smaller than this value are considered the same. The functor will write into a log file the arrays received as parameter, which numbers were found identical and which ones were removed. The functor will count how many numbers were removed and provide a query so the

clients can retrieve this counter.

- Create a functor (a function object) that checks if a username/password combination is valid. The functor will look into a file (a simulated database) searching for the username/password combination: if it's found, then the function object should return true, false otherwise. Upon instantiation, the client should specify a filename; if the file doesn't exist, the constructor should throw an exception.

## Lambda Expressions

- Create a lambda expression that calculates the sum of an array of numbers received as parameter; return the sum to the client.
- Create a lambda expression that trims a string (removes the spaces from beginning/end of the string). The string is received as parameter; the result will be stored in this parameter.
- Create a lambda expression that checks if all values from an array of numbers are in the interval `[min, max]`. The interval is captured by value from the context.
- Create a lambda expression that checks if a string contains only lower case letters. Return true if yes, false otherwise.
- Create a lambda expression that captures from the context a numeric value, receives as parameter an array of numbers and multiplies each element of the array with the number captured from the context.
- Create a lambda expression that encrypts/decrypts a string (encryption = add `n` to the ASCII code of each character, where `n` is captured from the context). The client should specify as parameters the string and the type of operation (an enumeration: encrypt/decrypt).

## Classes (rule-of-5, type members)

- Create a family of class named `collection` (a template class) that manages a dynamically allocated collection of items of any type. The class should support the following operations:
  - create an empty collection
  - rule-of-5
  - add an item
  - remove an item
  - retrieve the item at a specified index
  - modify the item from a specified index
  - print the content of the collection to screen

..

- remove all items
- remove all items that match a certain condition (a lambda expression received from the client as a parameter)
- create a new collection that contains only items that match a certain condition (a lambda expression received from the client as a parameter)

The class should use exceptions to inform the client when an operation cannot be completed (e.g., bad parameters)

- You are provided with the following code

```
enum class ItemType
{
    Clothing,
    Electronics,
    Appliance,
    Book,
    Movie,
    Computers,
    Tools,
    Gaming,
    Sports
}
class Item
{
    const ItemType m_category;
    std::string m_description{};
    std::string m_id{};
    double m_price{};
    double m_discount{}; // a number in the interval [0, 1]
    // ...
}
class ShoppingCart
{
    Item* m_pItems{}; // a dynamically allocated array of items in the shopping cart
    size_t m_cnt{};
    // ...
}
```

Add to the two classes members to support the following operations:

- ShoppingCart
- add items to the shopping cart
- remove items from the shopping cart
- rule-of-5 for the shopping cart
- remove from the shopping cart all items from a specified category (e.g., `ItemType::Tools`)
- remove from the shopping cart all items that don't have a discount (discount is 0)
- update the discount on all items in a shopping cart that belong to a certain category (e.g.,

ItemType::Book )

- remove from the shopping cart all items that match a predicate received as a parameter (this is a lambda expression)